



Tensor Decomposer: Web-Based Multidimensional Analysis Platform

Presenter

Md. Kamrul Hasan
BSSE-1418

Supervisor

Mohd. Zulfiquar Hafiz

Professor

**Institute of Information
Technology**

University of Dhaka

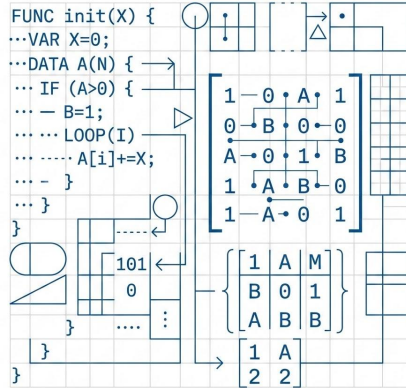
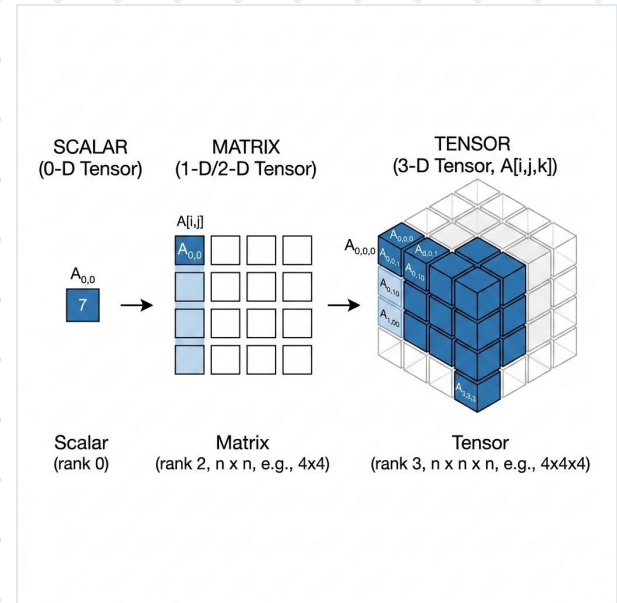


FIG. 01 // SYSTEM MODEL

Tensors: Multidimensional Data Arrays

Core Concept & Applications

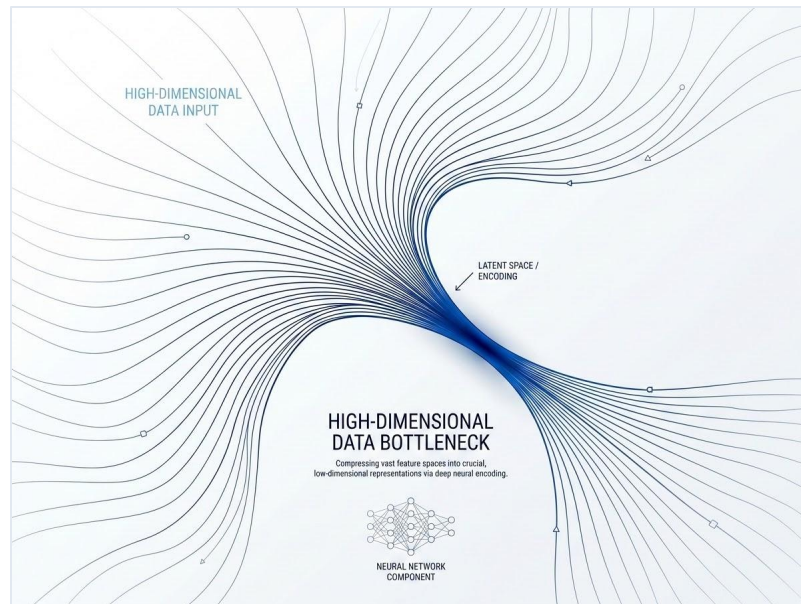
- **Mathematical Framework:** Generalization of scalars (0D), vectors (1D), and matrices (2D).
- **N-Dimensional Arrays:** Represent complex, multi-way relationships across multiple axes.
- **Common Applications:** Powering RGB images, high-dimensional video streams, and hypergraph data structures.



The Curse of Dimensionality

SYSTEM LIMITATIONS & BOTTLENECKS

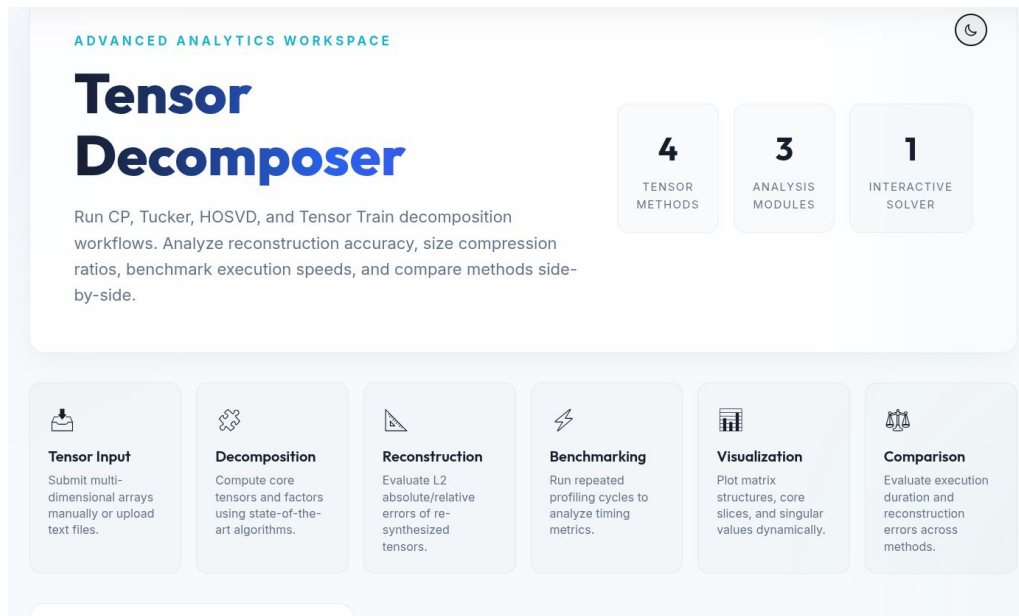
- Exponential memory growth with each added dimension.
- Standard matrix operations become computationally bottlenecked.
- Lack of accessible, interactive web tools for real-time tensor analysis.



Web-Based Tensor Decomposer

Platform Benefits

- A browser-native platform for tensor factorization.
- Bridges advanced linear algebra with an accessible UI.
- Enables real-time parameter tuning and immediate visual feedback.



The screenshot displays the 'Tensor Decomposer' web interface within an 'ADVANCED ANALYTICS WORKSPACE'. The main title 'Tensor Decomposer' is prominently displayed in a large, bold, blue font. Below the title, a descriptive paragraph states: 'Run CP, Tucker, HOSVD, and Tensor Train decomposition workflows. Analyze reconstruction accuracy, size compression ratios, benchmark execution speeds, and compare methods side-by-side.' To the right of the main content, there are three numbered steps in a vertical sequence: '4 TENSOR METHODS', '3 ANALYSIS MODULES', and '1 INTERACTIVE SOLVER'. At the bottom of the interface, there is a horizontal row of six functional modules, each with an icon and a brief description: 'Tensor Input' (submit/upload files), 'Decomposition' (compute core tensors/factors), 'Reconstruction' (evaluate L2 errors), 'Benchmarking' (run profiling cycles), 'Visualization' (plot matrix structures), and 'Comparison' (evaluate execution duration/errors).

ADVANCED ANALYTICS WORKSPACE

Tensor Decomposer

Run CP, Tucker, HOSVD, and Tensor Train decomposition workflows. Analyze reconstruction accuracy, size compression ratios, benchmark execution speeds, and compare methods side-by-side.

4
TENSOR METHODS

3
ANALYSIS MODULES

1
INTERACTIVE SOLVER

Tensor Input
Submit multi-dimensional arrays manually or upload text files.

Decomposition
Compute core tensors and factors using state-of-the-art algorithms.

Reconstruction
Evaluate L2 absolute/relative errors of re-synthesized tensors.

Benchmarking
Run repeated profiling cycles to analyze timing metrics.

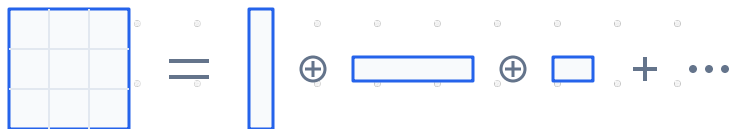
Visualization
Plot matrix structures, core slices, and singular values dynamically.

Comparison
Evaluate execution duration and reconstruction errors across methods.

CP Decomposition

CP Decomposition

- Factorizes a tensor into a sum of rank-one tensors.
- ALS Method



Tucker Decompositions

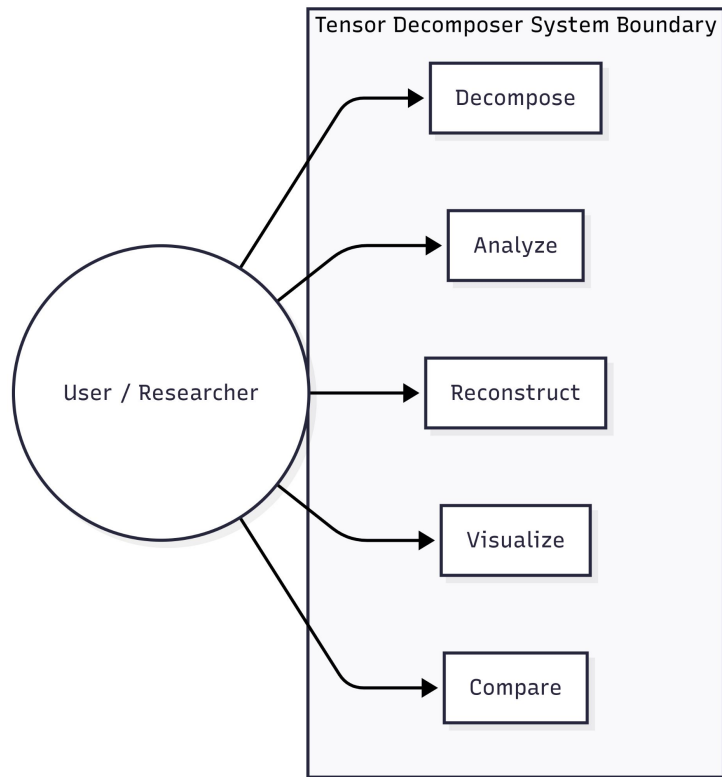


Tucker Decomposition

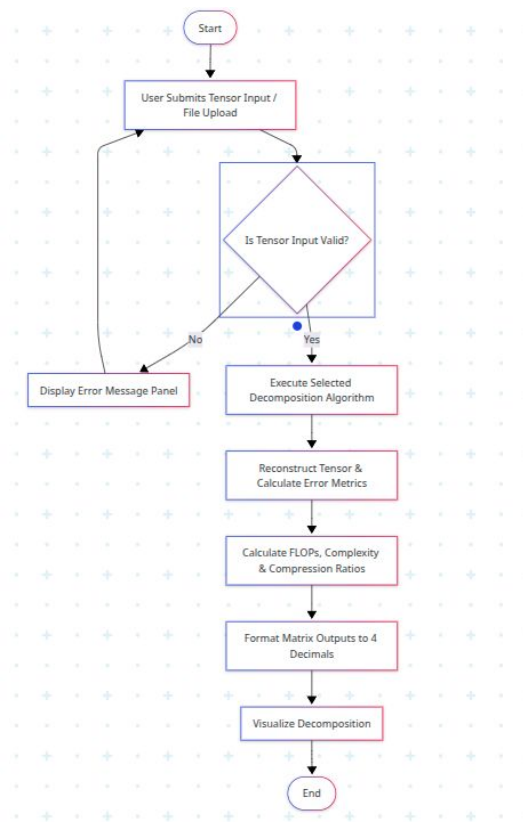
- A form of higher-order PCA.
- Extracts a core tensor and corresponding factor matrices.
- HOOI Method



Architecture: Use Case



Architecture: Activity



Tensor Decomposition

Tucker Decomposition:

- Decomposes a tensor into a core tensor multiplied by a matrix along each mode.

CP Decomposition (CANDECOMP/PARAFAC):

- Expresses a tensor as a sum of a minimum number of rank-one tensors.

Configuration

Define input tensor parameters and algorithm options

TENSOR INPUT (JSON ARRAY)

```
[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
```

UPLOAD TENSOR FILE

dense_te..._3_3_3.txt

METHOD / ALGORITHM

CP Decomposition

Decomposition Factors

[Download JSON](#)

Algorithm Method: CP

Weights:

```
[ 2.9135, 0.4561, 0.6886 ]
```

Factors:

Slice [0, :, :]:

```
[ [ 0.6019, 0.5022, -0.4752 ], [ 0.6265, -0.1966, -0.7855 ], [ 0.4953, 0.8421, -0.3965 ] ]
```

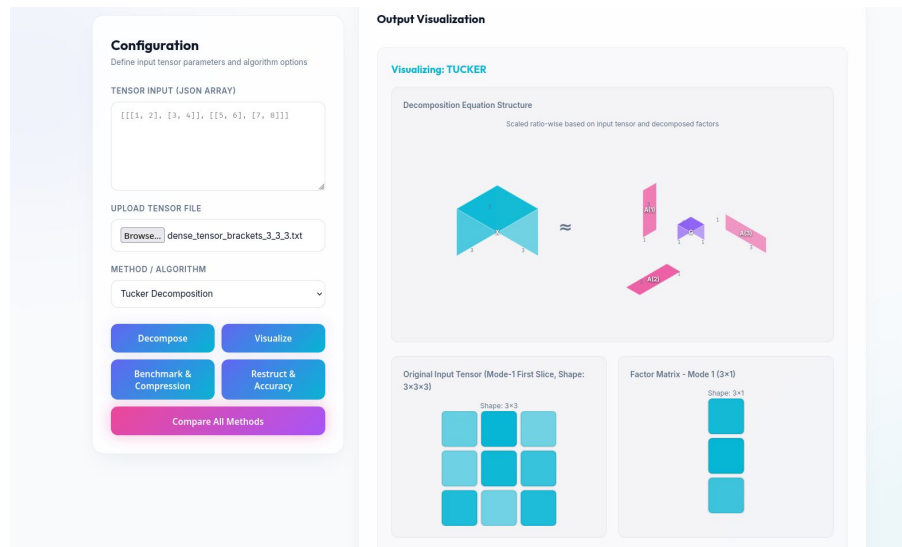
Slice [1, :, :]:

```
[ [ -0.5882, 0.4472, 0.9400 ], [ -0.6153, 0.1241, 0.3241 ], [ 0.8810, 0.8088, 0.8823 ] ]
```

SVG Visualizer

Vector Engine Capabilities

- Interactive 3D representations of tensor structures.
- Drag-and-drop data upload with immediate visual validation.



Performance Analysis

Accuracy

Accuracy calculated by relative, mean and absolute value difference from original tensor and reconstructed tensor

Configuration

Define input tensor parameters and algorithm options

TENSOR INPUT (JSON ARRAY)

```
[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
```

UPLOAD TENSOR FILE

Choose File dense_te...3_3_3.txt

METHOD / ALGORITHM

Tucker Decomposition

Decompose Visualize

Benchmark & Compression Restruct & Accuracy

Compare All Methods

Accuracy Metrics

MEAN ABSOLUTE ERROR (MAE)

0.1961

ROOT MEAN SQUARED ERROR (RMSE)

0.2437

RELATIVE ERROR

0.4098

RECONSTRUCTED MATRIX ELEMENT HEAD (MIN(10, LEN(TENSOR)))

```
{
  "algorithm": "tucker",
  "original_parameters": 27,
  "compressed_parameters": 10,
  "compression_ratio": 2.7,
  "absolute_error": 1.2661,
  "mean_absolute_error": 0.1961,
  "root_mean_squared_error": 0.2437,
  "relative_error": 0.4098,
  "reconstructed_head": [
```

Benchmark

FLOPs PERFORMANCE

Decompositon Efficiency

Floating-point operations analyzed across core decomposition pipelines to ensure minimal computational complexity during dense tensor operations.

Configuration

Define input tensor parameters and algorithm options

TENSOR INPUT (JSON ARRAY)

```
[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
```

UPLOAD TENSOR FILE

Choose File dense_te...3_3_3.txt

METHOD / ALGORITHM

Tucker Decomposition

Decompose

Visualize

Benchmark
&
Compression

Restruct &
Accuracy

Performance Analysis

EXECUTION TIME

38.695 ms

ESTIMATED FLOPS

40.50 KFLOPs

COMPLEXITY FORMULA

$O(N_1 N_2 N_3 R \cdot \text{Iterations})$

COMPRESSION RATIO

0.9x

PARAMETERS (COMPRESSED / ORIG)

30 / 27

RUNS MEASURED

1

Performance Metrics: CP

Execution Metrics

Execution
Time

38.695
ms

Computational
FLOPs

40.50
KFLOPs

Compare Methods

MULTI-METHOD ANALYSIS

Algorithmic Trade-offs

Direct side-by-side performance benchmarking of execution times, FLOPs complexity, and compression ratios across different core tensor decomposition pipelines.

Configuration

Define input tensor parameters and algorithm options

TENSOR INPUT (JSON ARRAY)

[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]

UPLOAD TENSOR FILE

dense_tensor_brackets_3_3_3.txt

METHOD / ALGORITHM

CP Decomposition

Decompose

Visualize

Benchmark &
Compression

Restruct &
Accuracy

Compare All Methods

Algorithm Comparison Summary

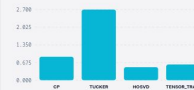
METHOD	COMPRESSION RATIO	RELATIVE ERROR	EXECUTION TIME
CP	0.9x	0.477101	31.239 ms
TUCKER	2.7x	0.409832	0.887 ms
HOSVD	0.5x	0	0.142 ms
TENSOR_TRAIN	0.6x	0	0.085 ms

Method Comparison Visualizations

Relative Reconstruction Error (Lower is Better)



Compression Ratio (Higher is Better)



Execution Time (ms) (Lower is Better)



Remaining Work

- **Current Challenge:** HOSVD and Tensor Train algorithm left to implement
- **Future:** Further Optimization on each algorithm
- **Extension:** Developing a standalone local Python library for offline use.

References

- **Kolda, T. G., & Bader, B. W. (2009).** Tensor decompositions and applications. SIAM Review, 51(3), 455-500.
- **Tucker, L. R. (1966).** Some mathematical notes on three-mode factor analysis. Psychometrika, 31(3), 279-311.



TENSOR DECOMPOSER

Thank You